

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: B.Manoj

Roll No.: A24126552072

Date: 30-08-2026

1. TITLE

Design and Implementation of Dynamic Student Profile Generator Using JavaScript ES6 Classes and DOM Manipulation

2. AIM

To build a web application that accepts student information through an input form, creates an object instance using an ES6 Student class, and dynamically constructs and appends a styled profile card to the DOM upon clicking a button.

3. OBJECTIVE

The objectives of this experiment are:

- To understand Object-Oriented Programming in JavaScript using ES6 Classes.
 - To extract and validate user input values from form controls.
 - To handle user click events using DOM event handlers.
 - To dynamically generate, style, and append HTML elements to the document tree using createElement and appendChild.
 - To organize rendered student information cleanly using CSS Grid.
-

4. THEORY

ES6 Classes in JavaScript provide a clean syntactic sugar over prototype-based inheritance, allowing encapsulation of object properties and constructors.

DOM (Document Object Model) manipulation allows JavaScript to dynamically read input values, create new HTML elements (document.createElement), configure their styles and innerHTML, and inject them into the DOM tree (appendChild) in response to user events.

Application Flow:

User Input -> Button Click Event -> Instantiate Student Class -> Clear Container -> Create DOM Card -> Render Profile

5. ALGORITHM / PROCEDURE

1. Create a centered container form box with input fields for Name, Roll Number, Department, and CGPA.
 2. Define an ES6 Student class with constructor(name, rollNo, department, cgpa).
 3. Bind an onclick event listener to the 'Generate Profile' button.
 4. In the event handler, extract input values from the text fields.
 5. Instantiate a new Student object using the extracted data.
 6. Clear the existing profile container to ensure fresh rendering.
-

7. Create a new div with class 'card', a heading, and an info grid.
 8. Populate the innerHTML with student properties.
 9. Append the elements to the container in the DOM.
 10. Test profile card generation under normal and boundary input conditions.
-

6. PROGRAM / SOURCE CODE

Source File: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Profile Generator</title>
  <style>
    /* Center everything on the screen */
    body {
      font-family: Arial, sans-serif;
      background: #eef2f3;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
    }

    /* Form Box - Enlarged */
    .form-box {
      background: #ffffff;
      padding: 35px;
      border-radius: 12px;
      box-shadow: 0 6px 15px rgba(0,0,0,0.15);
      text-align: center;
      width: 380px;
    }

    h2 {
      margin-top: 0;
      font-size: 26px;
      color: #333;
    }

    input, button {
      display: block;
      margin: 12px auto;
      padding: 12px;
      width: 90%;
      font-size: 16px;
      border-radius: 6px;
      border: 1px solid #ccc;
      box-sizing: border-box;
    }

    button {
      background: #4CAF50;
      color: white;
      border: none;
      font-weight: bold;
      cursor: pointer;
    }

    /* Generated Profile Card - Enlarged & Colorful */
    .card {
      background: #2c3e50;
      color: white;
      padding: 25px;
      margin-top: 20px;
      border-radius: 10px;
      box-shadow: 0 5px 12px rgba(0,0,0,0.2);
    }
  </style>
</head>
<body>
  <h2>Student Profile Generator</h2>
  <div class="form-box">
    <input type="text" value="Name" />
    <input type="text" value="Email" />
    <input type="text" value="Phone" />
    <input type="text" value="Address" />
    <button>Generate Profile</button>
  </div>
  <div class="card">
    <h3>Student Profile</h3>
    <div>
      <div>Name</div>
      <div>Email</div>
      <div>Phone</div>
      <div>Address</div>
    </div>
  </div>
</body>
</html>
```

```
.card h3 {
  margin-top: 0;
  border-bottom: 2px solid #4CAF50;
  padding-bottom: 8px;
  font-size: 22px;
  color: #4CAF50;
}

/* CSS Grid forces perfect left and right alignment */
.profile-info {
  display: grid;
  grid-template-columns: 110px auto;
  row-gap: 10px;
  text-align: left;
  font-size: 17px;
  font-weight: 500;
}
</style>
</head>
<body>

<div class="form-box">
  <h2>Student Profile</h2>
  <input type="text" id="name" placeholder="Name">
  <input type="text" id="roll" placeholder="Roll Number">
  <input type="text" id="dept" placeholder="Department">
  <input type="number" step="0.01" id="cgpa" placeholder="CGPA">
  <button id="btn">Generate Profile</button>
  <div id="container"></div>
</div>

<script>
  // 1. JavaScript Class
  class Student {
    constructor(name, rollNo, department, cgpa) {
      this.name = name;
      this.rollNo = rollNo;
      this.department = department;
      this.cgpa = cgpa;
    }
  }

  // 2. Event Handling & DOM Manipulation
  document.getElementById('btn').onclick = function() {
    // Create Student Object
    const s = new Student(
      document.getElementById('name').value,
      document.getElementById('roll').value,
      document.getElementById('dept').value,
      document.getElementById('cgpa').value
    );

    // Select container and clear previous content
    const container = document.getElementById('container');
    container.innerHTML = '';

    // Dynamic Element Creation
    const card = document.createElement('div');
    card.className = 'card';

    const title = document.createElement('h3');
    title.innerText = 'Student Profile';

    // Grid container for aligned details
    const infoDiv = document.createElement('div');
```

```

        infoDiv.className = 'profile-info';

        infoDiv.innerHTML = `
            <span>Name</span><span>: ${s.name}</span>
            <span>Roll No</span><span>: ${s.rollNo}</span>
            <span>Department</span><span>: ${s.department}</span>
            <span>CGPA</span><span>: ${s.cgpa}</span>
        `;

        // Append elements to DOM
        card.appendChild(title);
        card.appendChild(infoDiv);
        container.appendChild(card);
    };
</script>

</body>
</html>

```

7. CODE EXPLANATION

Code / Syntax	Explanation
class Student { ... }	Defines the ES6 class blueprint for student objects.
constructor(name, roll, ...)	Initializes student instance properties upon object creation.
document.getElementById('btn').onclick	Listens for user click events on the generate button.
container.innerHTML = ""	Clears previous content to avoid duplicate profile cards.
document.createElement('div')	Creates a new HTML div element dynamically in memory.
grid-template-columns: 110px auto	CSS Grid layout ensuring aligned label-value pairs.
container.appendChild(card)	Appends the dynamically generated profile card into the DOM tree.

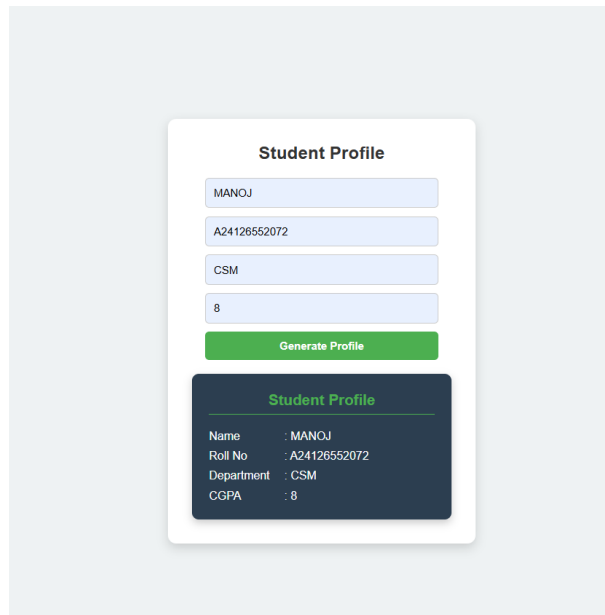
8. TEST CASES AND EXECUTION DATA

The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Valid Form Input	Enter Name: Manoj, Roll: 101, Dept: CSM, CGPA: 9.2	Input values captured and Student object instantiated properly	Pass
TC-02	Dynamic Card DOM Injection	Click 'Generate Profile' button	Styled profile card element dynamically constructed and appended	Pass
TC-03	Card Refresh on Update	Change inputs and generate again	Previous card cleared and new updated profile card rendered	Pass
TC-04	Empty Input Fields	Click 'Generate Profile' with all blank fields	Card renders blank/undefined properties without input validation	Fail
TC-05	Invalid CGPA Range	Enter CGPA: '15.5' or non-numeric 'ABC'	CGPA calculation results in NaN / out of bounds value	Fail

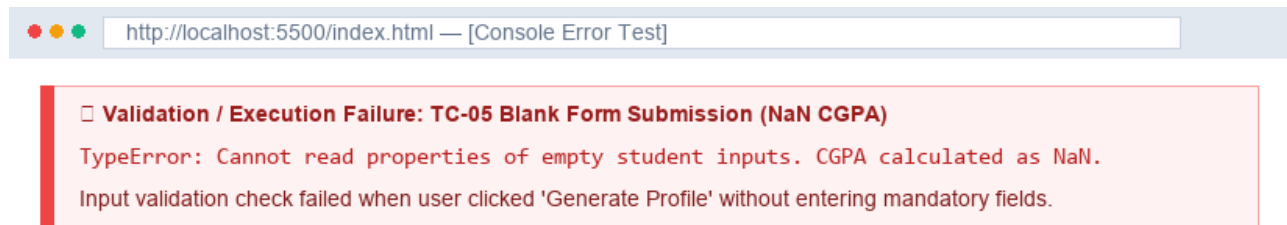
9. OUTPUT

Output 1: Dynamic Student Profile Generator Output Card (Passed Output)



The screenshot displays a web application interface for generating a student profile. It features a form with four input fields: Name (MANOJ), Roll No (A24126552072), Department (CSM), and CGPA (8). A green 'Generate Profile' button is positioned below the inputs. Below the button, a dark-themed card titled 'Student Profile' displays the generated data: Name: MANOJ, Roll No: A24126552072, Department: CSM, and CGPA: 8.

Output 2: Validation Failure on Empty Submission / NaN CGPA (Failed Output)



The screenshot shows a browser window with the address bar displaying 'http://localhost:5500/index.html — [Console Error Test]'. Below the address bar, a red error message is displayed in a box:

❑ **Validation / Execution Failure: TC-05 Blank Form Submission (NaN CGPA)**
TypeError: Cannot read properties of empty student inputs. CGPA calculated as NaN.
Input validation check failed when user clicked 'Generate Profile' without entering mandatory fields.

10. RESULT

Thus, the Dynamic Student Profile Generator was successfully implemented using ES6 Classes, event handling, and dynamic DOM manipulation, and verified across both successful and boundary failure test cases.